

PL / SQL EXERCISES

EXERCISE 1: CONTROL STRUCTURES

TABLE CREATION AND INSERT VALUES

QUERY

```
CREATE TABLE customers (  
    cust_id NUMBER PRIMARY KEY,  
    cname VARCHAR2(50),  
    age NUMBER,  
    balance NUMBER,  
    is_vip VARCHAR2(5) DEFAULT 'FALSE'  
);  
  
CREATE TABLE loans (  
    loan_id NUMBER PRIMARY KEY,  
    cust_id NUMBER,  
    int_rate NUMBER,  
    due_date DATE  
);  
  
INSERT INTO customers VALUES (1, 'Divya', 25, 15000, 'FALSE');  
INSERT INTO customers VALUES (2, 'Uma', 45, 18000, 'FALSE');  
INSERT INTO customers VALUES (3, 'Navashri', 20, 2000, 'FALSE');  
INSERT INTO customers VALUES (4, 'Tarun', 28, 55500, 'FALSE');  
INSERT INTO customers VALUES (5, 'Muthu', 65, 20000, 'FALSE');  
  
INSERT INTO loans VALUES (101, 1, 12, SYSDATE + 10);  
INSERT INTO loans VALUES (102, 2, 10, SYSDATE + 45);  
INSERT INTO loans VALUES (103, 3, 11, SYSDATE + 5);  
INSERT INTO loans VALUES (104, 4, 9, SYSDATE + 25);  
INSERT INTO loans VALUES (105, 5, 13, SYSDATE + 60);  
  
COMMIT;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
SQL> INSERT INTO customers VALUES (1, 'Divya', 25, 15000, 'FALSE')				
1 row inserted.				
Elapsed: 00:00:00.020				
SQL> INSERT INTO customers VALUES (2, 'Uma', 45, 18000, 'FALSE')				
1 row inserted.				

SCENARIO 1 — LOAN DISCOUNT FOR AGE > 60

QUERY

BEGIN

FOR cust_rec IN (

SELECT cust_id, age FROM customers

) LOOP

IF cust_rec.age > 60 THEN

UPDATE loans

SET int_rate = int_rate - 1

WHERE cust_id = cust_rec.cust_id;

DBMS_OUTPUT.PUT_LINE('Discount applied for: ' || cust_rec.cust_id);

END IF;

END LOOP;

COMMIT;

END;

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
				
<pre>SQL> DECLARE CURSOR c1 IS SELECT cust_id, age FROM customers; v_id NUMBER; v_age NUMBER;... Show more...</pre> Discount applied for: 5 PL/SQL procedure successfully completed. Elapsed: 00:00:00.165				

SCENARIO 2 — VIP STATUS FOR BALANCE > 10000

QUERY

BEGIN

FOR cust_rec IN (

SELECT cust_id, balance FROM customers

) LOOP

IF cust_rec.balance > 10000 THEN

UPDATE customers

SET is_vip = 'TRUE'

WHERE cust_id = cust_rec.cust_id;

DBMS_OUTPUT.PUT_LINE('VIP set for: ' || cust_rec.cust_id);

END IF;

END LOOP;

COMMIT;

END;

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 	<pre>CURSOR c2 IS SELECT cust_id, balance FROM customers; v_id NUMBER; v_bal NUMBER;... Show more...</pre> VIP set for: 1 VIP set for: 2 VIP set for: 4 VIP set for: 5 PL/SQL procedure successfully completed. Elapsed: 00:00:00.016			

SCENARIO 3 — LOAN DUE REMINDER (NEXT 30 DAYS)

QUERY

BEGIN

```
FOR loan_rec IN (
    SELECT l.loan_id, l.due_date, c.cname
    FROM loans l JOIN customers c
    ON l.cust_id = c.cust_id
    WHERE l.due_date <= SYSDATE + 30
) LOOP
    DBMS_OUTPUT.PUT_LINE('Reminder: ' || loan_rec.cname ||
        ' - Loan ' || loan_rec.loan_id ||
        ' due on ' || TO_CHAR(loan_rec.due_date, 'DD-MON-YYYY'));
END LOOP;
END;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
<pre>SQL> DECLARE CURSOR c3 IS SELECT l.loan_id, l.due_date, c.cname FROM loans l JOIN customers c... Show more...</pre> Reminder: Divya - Loan 101 due on 08-JUL-2026 Reminder: Navashri - Loan 103 due on 03-JUL-2026 Reminder: Tarun - Loan 104 due on 23-JUL-2026 PL/SQL procedure successfully completed. Elapsed: 00:00:00.022				

EXERCISE 3: STORED PROCEDURES

TABLE CREATION AND INSERT VALUES

QUERY

```
CREATE TABLE savings (
```

```
    acc_id NUMBER PRIMARY KEY,
```

```
    cname VARCHAR2(50),
```

```
    bal NUMBER
```

```
);
```

```
CREATE TABLE emp (
```

```
    eid NUMBER PRIMARY KEY,
```

```
    ename VARCHAR2(50),
```

```
    dept VARCHAR2(30),
```

```
    sal NUMBER
```

```
);
```

```
CREATE TABLE accounts (
```

```
    acc_no NUMBER PRIMARY KEY,
```

```
    cname VARCHAR2(50),
```

```
    amt NUMBER
```

```
);
```

```
INSERT INTO savings VALUES (1, 'Divya', 5000);
```

```
INSERT INTO savings VALUES (2, 'Uma', 8000);
```

```
INSERT INTO savings VALUES (3, 'Navashri', 3000);
```

```
INSERT INTO savings VALUES (4, 'Tarun', 12000);
```

```
INSERT INTO savings VALUES (5, 'Muthu', 6000);
```

```
INSERT INTO emp VALUES (1, 'Ravi', 'IT', 40000);
```

```
INSERT INTO emp VALUES (2, 'Kavi', 'IT', 35000);
```

```
INSERT INTO emp VALUES (3, 'Siva', 'HR', 30000);
```

```
INSERT INTO emp VALUES (4, 'Guna', 'HR', 28000);
```

```
INSERT INTO emp VALUES (5, 'Mani', 'IT', 45000);
```

```
INSERT INTO accounts VALUES (101, 'Divya', 10000);
```

```
INSERT INTO accounts VALUES (102, 'Uma', 5000);
```

```
INSERT INTO accounts VALUES (103, 'Navashri', 8000);
```

```
COMMIT;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
Table ACCOUNTS created. Elapsed: 00:00:00.017				
SQL> INSERT INTO savings VALUES (1, 'Divya', 5000) 1 row inserted. Elapsed: 00:00:00.018				
SQL> INSERT INTO savings VALUES (2, 'Uma', 8000)				

SCENARIO 1 — PROCESS MONTHLY INTEREST

QUERY

```
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest IS

BEGIN

    FOR acc_rec IN (

        SELECT acc_id, bal FROM savings

    ) LOOP

        UPDATE savings

        SET bal = bal + (bal * 0.01)

        WHERE acc_id = acc_rec.acc_id;

        DBMS_OUTPUT.PUT_LINE('Interest added for acc: ' || acc_rec.acc_id);

    END LOOP;

    COMMIT;

END;

//Calling Procedure:

BEGIN

    ProcessMonthlyInterest;

END;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
<pre>SQL> EXEC ProcessMonthlyInterest Interest added for acc: 1 Interest added for acc: 2 Interest added for acc: 3 Interest added for acc: 4 Interest added for acc: 5 PL/SQL procedure successfully completed. Elapsed: 00:00:00.018</pre>				

SCENARIO 2 — UPDATE EMPLOYEE BONUS

QUERY

```
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_dept VARCHAR2,
    p_bonus NUMBER
) IS
BEGIN
    FOR emp_rec IN (
        SELECT eid, sal FROM emp
        WHERE dept = p_dept
    ) LOOP
        UPDATE emp
        SET sal = sal + (sal * p_bonus / 100)
        WHERE eid = emp_rec.eid;
        DBMS_OUTPUT.PUT_LINE('Bonus updated for emp: ' || emp_rec.eid);
    END LOOP;
    COMMIT;
END;

//Calling Procedure:
BEGIN
    UpdateEmployeeBonus('IT', 10);
END;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
Elapsed: 00:00:00.031				
<pre>SQL> EXEC UpdateEmployeeBonus('IT', 10) Bonus updated for emp: 1 Bonus updated for emp: 2 Bonus updated for emp: 5 PL/SQL procedure successfully completed. Elapsed: 00:00:00.020</pre>				

SCENARIO 3 — TRANSFER FUNDS

QUERY

```
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_from NUMBER,
    p_to NUMBER,
    p_amt NUMBER
) IS
    v_bal NUMBER;
BEGIN
    SELECT amt INTO v_bal
    FROM accounts WHERE acc_no = p_from;
    IF v_bal < p_amt THEN
        DBMS_OUTPUT.PUT_LINE('Insufficient balance in acc: ' || p_from);
    ELSE
        UPDATE accounts SET amt = amt - p_amt WHERE acc_no = p_from;
        UPDATE accounts SET amt = amt + p_amt WHERE acc_no = p_to;
        COMMIT;
```

```
DBMS_OUTPUT.PUT_LINE('Transferred ' || p_amt || ' from acc ' || p_from || ' to acc ' ||  
p_to);
```

```
END IF;
```

```
END;
```

```
//Calling Procedure:
```

```
BEGIN
```

```
TransferFunds(101, 102, 2000);
```

```
TransferFunds(102, 103, 1000);
```

```
TransferFunds(101, 103, 500);
```

```
END;
```

OUTPUT

Query result	Script output	DBMS output	Explain Plan	SQL history
 				
Procedure TRANSFERFUNDS compiled Elapsed: 00:00:00.032				
SQL> EXEC TransferFunds(101, 102, 2000) Transfer done: 2000 from acc 101 to acc 102 PL/SQL procedure successfully completed. Elapsed: 00:00:00.156				